Lab02

姓名：杨梦琪
学号：25300120072

Phase01 关键汇编及恢复出的计算关系

p1_compute

Dump of assembler code for function p1_compute:

0x0000000000402050 <+0>: endbr64
0x0000000000402054 <+4>: push %rbp
0x0000000000402055 <+5>: mov %rdi,%rax 将 challenge 的完整值放于 rax
0x0000000000402058 <+8>: mov %rdi,%rdx 将 challenge 的值放于 rdx
0x000000000040205b <+11>: mov %edi,%esi 将 challenge 的低 32 位值放于 esi
0x000000000040205d <+13>: shr $0x20,%rax 将 challenge 右移 32 位，并将高 32 位放入 rax
0x0000000000402061 <+17>: mov %edi,%r8d 将 challenge 的低 32 位放入 r8d
0x0000000000402064 <+20>: shr $0x10,%rdx 将 challenge 的值右移 16 位
0x0000000000402068 <+24>: mov %eax,%edi edi=rax，放入高 32 位值
0x000000000040206a <+26>: mov %rsp,%rbp

Dump of assembler code for function p1_step2:

0x0000000000402040 <+0>: endbr64
0x0000000000402044 <+4>: lea (%rdi,%rsi,8),%eax 计算出 eax 的值等于 rdi+rsi*8
0x0000000000402047 <+7>: xor $0xf153,%eax 将 eax 和给出的立即数按位异或
0x000000000040204c <+12>: ret 返回 eax 的值

0x000000000040206d <+29>: call 0x402040 <p1_step2>调用这个函数，并将函数的返回值保存在 eax
0x0000000000402072 <+34>: mov %rdx,%rsi 设置 step0 的第二个参数，=challenge 右移 16 位的值
0x0000000000402075 <+37>: mov %r8d,%edi step0 的第一个参数 edi=challenge 的低 32 位
0x0000000000402078 <+40>: mov %eax,%ecx 将调用 step2 得出的 eax 存入 ecx

Dump of assembler code for function p1_step0:

0x0000000000402020 <+0>: endbr64
0x0000000000402024 <+4>: lea (%rdi,%rdi,2),%eax 计算出 eax 的值等于 rdi+rdi*2
0x0000000000402027 <+7>: add $0x82c3,%esi 计算 esi=esi+0x82c3
0x000000000040202d <+13>: xor %esi,%eax 将 esi 和 eax 的值按位异或
0x000000000040202f <+15>: ret 返回 eax 的值

0x000000000040207a <+42>: call 0x402020 <p1_step0>调用 step0 函数，并将值存于 eax
0x000000000040207f <+47>: mov %ecx,%esi step1 的第二个参数将 esi=step2 返回结果

0x0000000000402081 <+49>: mov %eax,%edi step1 的第一个参数 edi 存储 step0 返回结果

Dump of assembler code for function p1_step1:

0x0000000000402030 <+0>: endbr64

0x0000000000402034 <+4>: xor $0x310b,%esi 将 esi 的值和 0x310b 按位异或

0x000000000040203a <+10>: lea (%rsi,%rdi,4),%eax 计算 eax=rsi+rdi*4

0x000000000040203d <+13>: ret 返回 eax 的值

0x0000000000402083 <+51>: call 0x402030 <p1_step1> 调用函数 step1，并将值返回 eax

0x0000000000402088 <+56>: pop %rbp

0x0000000000402089 <+57>: ret 返回值 eax，即最终解出来的口令

推理过程

这道题目针对 challenge 这个数字进行处理，分别存储了其高 32 位、低 32 位、challenge 右移 16 位；存储后，又依次调用 step2、step0、step1 三个函数。其中 step2 调用了高 32 位和低 32 位，step0 调用了低 32 位和右移 16 位的 challenge，step1 调用了 step2 和 step0 计算出来的结果，同时可得出最终答案

使用公开列表以外的 seed 验证的记录



已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

Phase02 关键汇编即恢复出的计算关系

Dump of assembler code for function phase_02:

0x000000000040215a <+106>: mov %r13,%rax 将 challenge 的值放入 rax 中

0x000000000040215d <+109>: mov -0x40(%rbp),%esi 取出第 1 参数放进 rsi

0x0000000000402160 <+112>: shr $0x2f,%rax 将 challenge 的值右移 47 位

0x0000000000402164 <+116>: movzbl %al,%eax 将右移后的 challenge 的低 8 位存在 eax 中

0x0000000000402167 <+119>: add $0x17,%eax eax=eax+23（十进制）

0x000000000040216a <+122>: cmp %eax,%esi 比较 eax 和 esi 的大小关系

0x000000000040216c <+124>: jne 0x402115 <phase_02+37> 不相等就爆炸

0x000000000040216e <+126>: mov $0x1,%edx 循环计数器，将十进制 1 的值存入 edx，意味着 i=1，从第二个数字开始循环

0x0000000000402173 <+131>: lea 0x2(%rdx),%ecx 将循环计数器的数字进行移位，移位量为 2 即 i+2

0x0000000000402176 <+134>: mov %r13,%rax 将 challenge 的值存在 rax 中

0x0000000000402179 <+137>: shr %cl,%rax challenge 的值右移 i+2 位

0x000000000040217c <+140>: lea (%rsi,%rsi,4),%ecx ecx=rsi+rsi*4

0x000000000040217f <+143>: mov -0x40(%rbp,%rdx,4),%esi esi=rbp+rdx*4-0x40, 即将 rsi 的值替换为输入的第 2 个参数

0x0000000000402183 <+147>: and $0x3f,%eax 将 0x3f 和 eax 进行与运算，0x3f 为 0011 1111，即只保留 eax 的低 6 位

0x0000000000402186 <+150>: add %edx,%eax eax=eax+edx;

0x0000000000402188 <+152>: xor %ecx,%eax ecx 和 eax 取异或

0x000000000040218a <+154>: cmp %eax,%esi 比较 eax 和 esi 的大小

0x000000000040218c <+156>: jne 0x402115 <phase_02+37> 不相等就爆炸

0x000000000040218e <+158>: add $0x1,%rdx 给循环计数器 rdx+1；

0x0000000000402192 <+162>: cmp $0x6,%rdx 比较 rdx 和 6 的大小关系

0x0000000000402196 <+166>: jne 0x402173 <phase_02+131>如果不相等，就返回+131 哪里重新开始循环

0x0000000000402198 <+168>: jmp 0x402117 <phase_02+39>4

推理过程

通过观察可发现，共有 6 个数字的地址被存入了寄存器，那我们就知道共有 6 个参数。

汇编中，从<+106>开始才是需要还原计算过程的汇编代码。前一部分主要是对 challenge 的值进行移位，然后才以寄存器 rdx 作为循环计数器，同时循环从 1 开始，也就利用 2-6 这 5 个参数分别进行过程相同的循环计算，计算出来的值保存到 value 中，循环 5 次。

使用公开列表以外的 seed 验证的记录

```
root@ics:~/ics-2026-bomblab# make
./defuser --seed 52 --phase 2 | ./bomb --seed 52 --phase 2
make: Nothing to be done for 'all'.
──────────────────────────────────────────────────────────
BOMBLAB 2026                                  这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP   \
                    |             |
                     \           /
                      '---------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x3c25a252481d41a2
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

Phase03 关键汇编即恢复出的计算关系

Dump of assembler code for function phase_03:

0x00000000004022ca <+58>: mov $0x403186,%esi 2 个有效参数
0x00000000004022c2 <+50>: lea -0x14(%rbp),%rcx
0x00000000004022c6 <+54>: lea -0x18(%rbp),%rdx
0x00000000004022ca <+58>: mov $0x403186,%esi
0x00000000004022cf <+63>: mov %rbx,%rdi
0x00000000004022d7 <+71>: cmp $0x2,%eax

```
   0x00000000004022dc <+76>:    mov    %r12d,%edi    将 edi 中放置 challenge 的低 32 位
   0x00000000004022e1 <+81>:    and    $0x7,%edi     仅保留 challenge 的低 3 位
   0x00000000004022e9 <+89>:    mov    %r12,%rax     将 challenge 的值存入 rax
   0x00000000004022ec <+92>:    mov    %r12,%rsi     将 challenge 的值存入 rsi
   0x00000000004022ef <+95>:    shr    $0x3,%rax     将 challenge 右移 3 位
   0x00000000004022f3 <+99>:    shr    $0x20,%rsi    将 challenge 右移 32 位
   0x00000000004022f7 <+103>:   xor    %eax,%esi     esi+esi^eax
Dump of assembler code for function p3_dispatch:
   0x0000000000402230 <+0>:     endbr64
   0x0000000000402234 <+4>:     mov    %edi,%eax     将 edi 存入 eax
   0x0000000000402236 <+6>:     mov    %esi,%edi     将 esi 存入 edi
   0x0000000000402238 <+8>:     cmp    $0x7,%eax     校验原始的 edi 是否超过 0~7 的范
围
   0x000000000040223b <+11>:    ja     0x402285 <p3_dispatch+85>超过则清零返回
   0x000000000040223d <+13>:    notrack jmp *0x405820(,%rax,8)这部分为寻址，具体地址
计算方式为 0x405820+rax*8,rax 是多少，就跳转到 case 多少
   0x0000000000402245 <+21>:    nopl   (%rax)  空指令
   0x0000000000402248 <+24>:    jmp    0x402210 <p3_case_6>
Dump of assembler code for function p3_case_6:
   0x0000000000402210 <+0>:     endbr64
   0x0000000000402214 <+4>:     lea    (%rdi,%rdi,1),%eax    eax=rdi*2
   0x0000000000402217 <+7>:     xor    $0xb01e,%eax eax=eax^0xb01e
   0x000000000040221c <+12>:    ret
   0x000000000040224a <+26>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402250 <+32>:    jmp    0x4021c0 <p3_case_2>
Dump of assembler code for function p3_case_2:
   0x00000000004021c0 <+0>:     endbr64
   0x00000000004021c4 <+4>:     lea    0x1772(%rdi,%rdi,4),%eax    eax=0x1772+rdi*5
   0x00000000004021cb <+11>:    ret
   0x0000000000402255 <+37>:    nopl   (%rax)
   0x0000000000402258 <+40>:    jmp    0x4021f0 <p3_case_4>
Dump of assembler code for function p3_case_4:
   0x00000000004021f0 <+0>:     endbr64
   0x00000000004021f4 <+4>:     lea    0x13768(%rdi,%rdi,4),%eax    eax=0x13768+rdi*5
   0x00000000004021fb <+11>:    ret
   0x000000000040225a <+42>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402260 <+48>:    jmp    0x4021d0 <p3_case_3>
Dump of assembler code for function p3_case_3:
   0x00000000004021d0 <+0>:     endbr64
   0x00000000004021d4 <+4>:     lea    0x0(,%rdi,4),%eax    eax=0x0+rdi*4
   0x00000000004021db <+11>:    xor    $0x894d,%eax    eax=eax^0x894d
   0x00000000004021e0 <+16>:    ret
   0x0000000000402265 <+53>:    nopl   (%rax)
   0x0000000000402268 <+56>:    jmp    0x402220 <p3_case_7>
```

```
Dump of assembler code for function p3_case_7:
   0x0000000000402220 <+0>:     endbr64
   0x0000000000402224 <+4>:     lea    0x25d6(%rdi,%rdi,2),%eax    eax=0x25d6+rdi*3
   0x000000000040222b <+11>:    ret
   0x000000000040226a <+58>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402270 <+64>:    jmp    0x4021b0 <p3_case_1>
Dump of assembler code for function p3_case_1:
   0x00000000004021b0 <+0>:     endbr64
   0x00000000004021b4 <+4>:     mov    %edi,%eax    将 edi 的值存入 eax
   0x00000000004021b6 <+6>:     shr    $1,%edi    将 edi 右移 1 位
   0x00000000004021b8 <+8>:     xor    $0x40eb,%eax    eax=0x40eb^eax
   0x00000000004021bd <+13>:    add    %edi,%eax    eax=edi+eax
   0x00000000004021bf <+15>:    ret
   0x0000000000402275 <+69>:    nopl   (%rax)
   0x0000000000402278 <+72>:    jmp    0x402200 <p3_case_5>
Dump of assembler code for function p3_case_5:
   0x0000000000402200 <+0>:     endbr64
   0x0000000000402204 <+4>:     mov    %edi,%eax    将 edi 的值存入 eax
   0x0000000000402206 <+6>:     shr    $1,%edi    将 edi 的值右移 1 位
   0x0000000000402208 <+8>:     xor    $0x7152,%eax    eax=eax^0x7152
   0x000000000040220d <+13>:    add    %edi,%eax    eax=edi+eax
   0x000000000040220f <+15>:    ret
   0x000000000040227a <+74>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402280 <+80>:    jmp    0x4021a0 <p3_case_0>
Dump of assembler code for function p3_case_0:
   0x00000000004021a0 <+0>:     endbr64
   0x00000000004021a4 <+4>:     lea    0x2ef90(%rdi,%rdi,4),%eax    eax=0x2ef90+rdi*5
   0x00000000004021ab <+11>:    ret
   0x0000000000402285 <+85>:    xor    %eax,%eax
   0x0000000000402287 <+87>:    ret

   0x00000000004022f9 <+105>:   call   0x402230 <p3_dispatch>
   0x00000000004022fe <+110>:   cmp    %eax,-0x14(%rbp)
   0x0000000000402301 <+113>:   sete   %al
   0x0000000000402304 <+116>:   jmp    0x4022b5 <phase_03+37>
```

推理过程

这道题目中有提示运用了 switch 函数，同时通过判断，本题共有两个参数，第一个参数是 num1，也就是计算出来的 challenge 低 3 位的值，通过这个 num1，来判断下一步会进入 switch 的哪一个分支，在这些分支下根据汇编进行运算即可。

大意了发现这样并不对，switch 里面的 case 和汇编代码跳转的这些 case 拥有一个特定顺序，运用 ai 的协助下，在终端输入 x/8xg 0x405820
找到这样一个跳转表：

```
(gdb) x/8xg 0x405820
0x405820:       0x0000000000402258      0x0000000000402260
0x405830:       0x0000000000402268      0x0000000000402270
0x405840:       0x0000000000402278      0x0000000000402280
0x405850:       0x0000000000402248      0x0000000000402250
```

这个跳转表不直接存 case 函数的入口，存的是汇编代码中 jmp 跳板指令的地址。每次 jump 会跳到他的目标地址，而目标地址的 jump 顺序对应的就是 case0—7 返回的 case 情况。图中第一个目标地址就是 p3_case_4 所在地址，则 case0：return 0x13768 + b * 5，以此类推即可。

使用公开列表以外的 seed 验证的记录

```
root@ics:~/ics-2026-bomblab# make
./defuser --seed 109080 --phase 3 | ./bomb --seed 109080 --phase 3
make: Nothing to be done for 'all'.
```

BOMBLAB 2026　　这里还没有明天



[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

```
Phase 3 challenge: 0xb860bdadf2e0b916
Input (unsigned decimal integers):
Phase 3 defused.
```

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]

已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

Phase04 关键汇编即恢复出的计算关系

Dump of assembler code for function phase_04:

0x00000000004023b6 <+54>: mov $0x403189,%esi

0x00000000004023c8 <+72>: mov %r12,%rcx 将 challenge 的值复制进 rcx
0x00000000004023cb <+75>: mov %r12,%rax 将 challenge 的值复制进 rax
0x00000000004023ce <+78>: mov %r12d,%edi 将 challenge 的低 32 位复制进 edi
0x00000000004023d1 <+81>: xor %esi,%esi esi=0
0x00000000004023d3 <+83>: shr $0x20,%rax 将 rax 右移 32 位
0x00000000004023d7 <+87>: shr $0x7,%rcx 将 rcx 右移 7 位
0x00000000004023db <+91>: and $0x7f,%edi edi=edi&&0x7f
0x00000000004023de <+94>: mov $0x7f,%edx 将 edx 的值改为 0x7f
0x00000000004023e3 <+99>: xor %eax,%ecx 取 eax 和 ecx 的异或
0x00000000004023e5 <+101>: call 0x402310 <p4_walk> 调用函数 p4_walk

Dump of assembler code for function p4_walk:
0x0000000000402310 <+0>: endbr64
0x0000000000402314 <+4>: mov %edi,%eax 将 edi 的值赋值给 eax
0x0000000000402316 <+6>: mov %edx,%edi 将 edx 的值赋值给 edi
0x0000000000402318 <+8>: sub %esi,%edi edi=edi-esi
0x000000000040231a <+10>: shr $1,%edi 将 edi 的值右移 1 位
0x000000000040231c <+12>: add %esi,%edi edi=esi+edi
0x000000000040231e <+14>: cmp %eax,%edi 比较 edi 和 eax 的大小
0x0000000000402320 <+16>: je 0x402370 <p4_walk+96> 如果相等直接执行第+96 行汇编
0x0000000000402322 <+18>: push %rbp
0x0000000000402323 <+19>: mov %rsp,%rbp
0x0000000000402326 <+22>: push %rbx
0x0000000000402327 <+23>: lea (%rdi,%rcx,1),%ebx ebx=rdi+rcx
0x000000000040232a <+26>: sub $0x8,%rsp
0x000000000040232e <+30>: cmp %edi,%eax 比较 edi 和 eax 的大小
0x0000000000402330 <+32>: jb 0x402350 <p4_walk+64> 小于则直接进入+64 行
0x0000000000402332 <+34>: lea 0x1(%rdi),%esi esi=0x1+rdi
0x0000000000402335 <+37>: mov %eax,%edi 将 eax 的值存入 edi
0x0000000000402337 <+39>: call 0x402310 <p4_walk> 再次递归此函数
0x000000000040233c <+44>: lea (%rax,%rax,4),%eax eax=rax*5
0x000000000040233f <+47>: xor %ebx,%eax eax=eax^ebx
0x0000000000402341 <+49>: mov -0x8(%rbp),%rbx
0x0000000000402345 <+53>: leave
0x0000000000402346 <+54>: xor $0x1a,%eax eax=eax^0x1a
0x0000000000402349 <+57>: ret 结束调用递归
0x000000000040234a <+58>: nopw 0x0(%rax,%rax,1)

```
0x0000000000402350 <+64>:    lea    -0x1(%rdi),%edx    edx=rdi-1
0x0000000000402353 <+67>:    mov    %eax,%edi    将 eax 的值存入 edi
0x0000000000402355 <+69>:    call   0x402310 <p4_walk>    重新递归
0x000000000040235a <+74>:    lea    (%rax,%rax,4),%eax    eax=5*rax
0x000000000040235d <+77>:    xor    %ebx,%eax    eax=ebx^eax
0x000000000040235f <+79>:    mov    -0x8(%rbp),%rbx
0x0000000000402363 <+83>:    leave
0x0000000000402364 <+84>:    xor    $0xa,%eax    eax=0xa^eax
0x0000000000402367 <+87>:    ret    回到上一层调用处
0x0000000000402368 <+88>:    nopl   0x0(%rax,%rax,1)
0x0000000000402370 <+96>:    lea    0xf7(%rdi),%eax    eax=0xf7+rdi
0x0000000000402376 <+102>:   xor    %ecx,%eax    eax=eax^ecx
0x0000000000402378 <+104>:   ret
```

推理过程

首先通过判断我们已知这道题只有一个返回值，同时在函数内不断重复引用函数，则判断出来这是一个递归函数。这个递归函数又分三个分支进行递归，即 a=b a>b a<b,在递归调用的过程中，第一次我出现了这样的问题，直接进行 return。但是这样操作就会出现以下的问题：无法进行递归调用后的计算，所以我需要将递归调用得到的值存储起来，放进 res 中，最后计算后 return res。

使用公开列表以外的 seed 验证的记录



已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

Phase05 关键汇编即恢复出的计算关系

Dump of assembler code for function phase_05:

0x00000000004024c0 <+80>: mov %r12,%rdi 将 challenge 的值复制进 rdi 中

0x00000000004024c3 <+83>: call 0x402400 <p5_traverse> 调用函数 p5_traverse

Dump of assembler code for function p5_traverse:

0x0000000000402400 <+0>: endbr64

0x0000000000402404 <+4>: mov %rdi,%rax 将 challenge 的值复制进 rax

0x0000000000402407 <+7>: mov %rdi,%rcx 将 challenge 的值复制进 rcx

0x000000000040240a <+10>: mov %rsi,%r8 将 rsi 的值复制进 r8

0x000000000040240d <+13>: and $0x7,%eax eax=0x7&eax，实际上是设置数组有 8 个元素

0x0000000000402410 <+16>: shr $0x3,%rcx rcx 右移三位

0x0000000000402414 <+20>: mov 0x405b60(,%rax,8),%rdx 这部分可以确定，这是一个有 8 个元素，每个元素占 8 字节的数组

0x000000000040241c <+28>: mov %rdi,%rax rax=rdi

0x000000000040241f <+31>: shr $0x20,%rdi rdi 右移 32 位，即 challenge 高 32 位

0x0000000000402423 <+35>: shr $0x6,%rax rax 右移 6 位

0x0000000000402427 <+39>: xor %edi,%eax eax=eax^edi

0x0000000000402429 <+41>: mov 0x8(%rdx),%esi addr=0x8+rdx，这里表明在结构体中，esi 读取 8 字节后的 4 字节数字，即存储的第一个数据，说明此前有一个 next 指针

0x000000000040242c <+44>: mov %ecx,%edi edi=ecx

0x000000000040242e <+46>: xor $0xeb6,%eax eax=0xeb6 ^eax

0x0000000000402433 <+51>: lea (%rax,%rax,2),%eax eax=rax*3

0x0000000000402436 <+54>: add 0xc(%rdx),%eax 读取结构体中的存储的第二个数据，并存出来 eax 中

0x0000000000402439 <+57>: mov %esi,(%r8) 把第一个数据写入指针指向的内存

0x000000000040243c <+60>: xor %esi,%eax eax=eax^esi

0x000000000040243e <+62>: and $0x3,%edi edi=edi&0x3

0x0000000000402441 <+65>: je 0x402460 <p5_traverse+96> 若 edi==0，直接跳

到结尾返回

0x0000000000402443 <+67>: mov $0x1,%ecx ecx=1

0x0000000000402448 <+72>: mov (%rdx),%rdx rdx=rdx->next

0x000000000040244b <+75>: lea (%rax,%rax,2),%eax eax=3*rax

0x000000000040244e <+78>: add $0x1,%ecx 循环计数器自增

0x0000000000402451 <+81>: mov 0x8(%rdx),%esi esi=第一个节点

0x0000000000402454 <+84>: add 0xc(%rdx),%eax eax=第二个节点

0x0000000000402457 <+87>: xor %esi,%eax eax=esi^eax

0x0000000000402459 <+89>: mov %esi,(%r8) 将第一个节点存入 r8

0x000000000040245c <+92>: cmp %ecx,%edi 比较 edi（总步数）和 ecx（循环计数器）的大小

0x000000000040245e <+94>: jae 0x402448 <p5_traverse+72>大于等于则跳转，继续循环

0x0000000000402460 <+96>: ret

推理过程

首先，通过“mov 0x405b60(,%rax,8),%rdx”判断出，每个元素占 8 字节，所以判断出这应该是个指针数组。接着从 mov 0x8(%rdx),%esi 中又判断出，加上之后的 0xc(%rdx)及(%rdx)判断出，他们从同一个起始地址开始找到了三个元素，第一个元素 8 字节，第二三个元素分别为 4 字节，那我们就得知这是一个结构体，结构体第一个元素为 next 指针，第二三个元素为数据。并且通过后面的汇编代码我们判断出来这是一个 while 循环函数。得知这是一个有 8 个元素的指针数组，遍历的每个指针中有有一个结构体，结构体内有两个元素。

使用公开列表以外的 seed 验证的记录



已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

Phase06 关键汇编即恢复出的计算关系

Dump of assembler code for function phase_06:

0x00000000004025e8 <+72>: mov %r12,%rdi 将 challenge 的值复制给 rdi

0x00000000004025eb <+75>: call 0x402550 <p6_dispatch>

Dump of assembler code for function p6_dispatch:

0x0000000000402550 <+0>: endbr64

0x0000000000402554 <+4>: push %rbp

0x0000000000402555 <+5>: mov %rsp,%rbp

0x0000000000402558 <+8>: push %rbx

0x0000000000402559 <+9>: mov %rdi,%rbx 将 rdi 的值复制给 rbx 保存

0x000000000040255c <+12>: shr $0x6,%rdi 将 rdi 的值右移 6 位

0x0000000000402560 <+16>: mov %rbx,%rax 将 rbx 的值复制给 rax 保存

0x0000000000402563 <+19>: shr $0x20,%rax 将 rax 的值右移 32 位

0x0000000000402567 <+23>: xor %eax,%edi edi=eax^edi

0x0000000000402569 <+25>: mov %rbx,%rax 将 rbx 的值复制给 rax

0x000000000040256c <+28>: sub $0x8,%rsp rsp=rsp-8

0x0000000000402570 <+32>: and $0x38,%ebx ebx=0x38&ebx

0x0000000000402573 <+35>: and $0x7,%eax eax=0x7&eax

0x0000000000402576 <+38>: call *0x405e00(,%rax,8) 跳转表，由<+35>得到下标 0~7，且每个表项占 8 字节

0x000000000040257d <+45>: xor $0x1940,%eax eax=0x1940^eax

0x0000000000402582 <+50>: mov %eax,%edi edi=eax

0x0000000000402584 <+52>: call *0x405e00(%rbx) 第二次跳转表，因为前面 rbx&0x38，所以取值只能是 8 及其倍数

0x000000000040258a <+58>: mov -0x8(%rbp),%rbx

0x000000000040258e <+62>: leave

0x000000000040258f <+63>: add $0x838,%eax eax=0x838+eax

0x0000000000402594 <+68>: ret

0x00000000004025f0 <+80>: cmp %eax,-0x14(%rbp)

0x00000000004025f3 <+83>: sete %al

0x00000000004025f6 <+86>: add $0x10,%rsp

0x00000000004025fa <+90>: pop %rbx

```
0x00000000004025fb <+91>:    pop    %r12
0x00000000004025fd <+93>:    pop    %
0x00000000004025fe <+94>:    ret
```

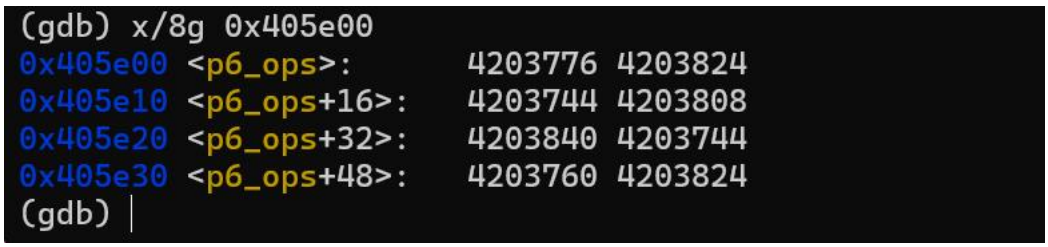
跳转表下标	函数地址	对应函数
0	0x402500	op2
1	0x402530	op4
2	0x4024e0	op0
3	0x402520	op3
4	0x402540	op5
5	0x4024e0	op0
6	0x4024f0	op1
7	0x402530	op4

```
Dump of assembler code for function p6_op_2:
   0x0000000000402500 <+0>:     endbr64
   0x0000000000402504 <+4>:     lea    0x0(,%rdi,4),%edx     edx=0+rdi*4
   0x000000000040250b <+11>:    lea    0x6dc0(%rdx),%eax     eax=0x6dco+rdx
   0x0000000000402511 <+17>:    add     %edi,%edx     edx=edi+edx
   0x0000000000402513 <+19>:    xor    %edx,%eax     eax=eax^edx
   0x0000000000402515 <+21>:    ret
End of assembler dump.
(gdb) disas 0x402530
Dump of assembler code for function p6_op_4:
   0x0000000000402530 <+0>:     endbr64
   0x0000000000402534 <+4>:     lea    0x8c30(%rdi),%eax    eax=0x8c30+rdi
   0x000000000040253a <+10>:    shr    $0x3,%edi    edi 右移 3 位
   0x000000000040253d <+13>:    xor    %edi,%eax    eax=edi^eax
   0x000000000040253f <+15>:    ret
End of assembler dump.
(gdb) disas 0x4024e0
Dump of assembler code for function p6_op_0:
   0x00000000004024e0 <+0>:     endbr64
   0x00000000004024e4 <+4>:     mov     %edi,%eax eax=edi
   0x00000000004024e6 <+6>:     xor    $0xdad3,%eax    eax=eax^0xdad3
   0x00000000004024eb <+11>:    lea    (%rax,%rdi,4),%eax    eax=rax+4*rdi
```

```
   0x00000000004024ee <+14>:    ret
End of assembler dump.
(gdb) disas 0x402520
Dump of assembler code for function p6_op_3:
   0x0000000000402520 <+0>:    endbr64
   0x0000000000402524 <+4>:    mov    %edi,%eax    eax=edi
   0x0000000000402526 <+6>:    shr    $1,%eax      eax=eax/2
   0x0000000000402528 <+8>:    xor    %edi,%eax    eax^=edi
   0x000000000040252a <+10>:   add    $0x23db,%eax     eax+=0x23db
   0x000000000040252f <+15>:   ret
End of assembler dump.
(gdb) disas 0x402540
Dump of assembler code for function p6_op_5:
   0x0000000000402540 <+0>:    endbr64
   0x0000000000402544 <+4>:    mov    %edi,%eax    eax=edi
   0x0000000000402546 <+6>:    ror    $0x7,%eax    将 eax 进行循环右移（相当于 eax
右移 7 位和左移 25 位进行或运算
   0x0000000000402549 <+9>:    ret
End of assembler dump.
(gdb) disas 0x4024f0
Dump of assembler code for function p6_op_1:
   0x00000000004024f0 <+0>:    endbr64
   0x00000000004024f4 <+4>:    mov    %edi,%eax    eax=edi
   0x00000000004024f6 <+6>:    rol    $0x3,%eax    eax 进行循环左移（相当于左移 3 位
和右移 29 位进行或运算）
   0x00000000004024f9 <+9>:    xor    $0xfac,%eax    eax^=0xfac
   0x00000000004024fe <+14>:   ret
```

推理过程

这道题目一个点就是他需要跳转到函数 p6_dispatch，这个函数实际上是一个计算函数，而通过 `call *0x405e00(,%rax,8) call *0x405e00(%rbx)` 这两行我们了解到他通过访问内存地址，而非函数名字跳转到一个函数，所以我们得出这是一个跳转表。由（ ,rax，8）我们得出内存中存的每个数据长度是 8 字节，所以我们就从地址 0x405e00 中以此读取所有 8 字节数据，即如截屏中所表示



接下来我们需要把这些十进制数字转换为十六进制，从最小到最大依次命名为 op0--0p5，按照这八个数字的顺序标注其跳转表下标，冰河函数名一一对应，即上表中所展示的。这样的话就可以跳转到相应函数，将 edi 的值带进函数进行运算。最后返回结果。

使用公开列表以外的 seed 验证的记录

```
root@ics:~/ics-2026-bomblab# make
./defuser --seed 20808 --phase 6 | ./bomb --seed 20808 --phase 6
make: Nothing to be done for 'all'.
```



已完成关卡的运行结果截图

```
root@ics:~/ics-2026-bomblab# python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
root@ics:~/ics-2026-bomblab#
```

Secret 关键汇编即恢复出的计算关系

Dump of assembler code for function secret_phase:

0x0000000000402620 <+0>: endbr64

0x0000000000402662 <+66>: lea -0x24(%rbp),%r8

0x0000000000402666 <+70>: lea -0x28(%rbp),%rcx

0x000000000040266f <+79>: lea -0x2c(%rbp),%rdx

```
   0x0000000000402680 <+96>:    mov    %r13d,%ecx      将 context 的值存入 ecx
   0x0000000000402683 <+99>:    xor    %edx,%edx     edx^=edx
   0x0000000000402685 <+101>:   and    $0x3,%ecx       ecx=ecx&0x3
   0x0000000000402688 <+104>:   cmp    %ecx,-0x2c(%rbp)       比较
   0x000000000040268b <+107>:   jne    0x40264a <secret_phase+42>

   0x000000000040268d <+109>:   mov    %r12,%rsi    将 challenge 的值存入 rsi
   0x0000000000402690 <+112>:   imul   $0xd597,%ecx,%r9d      r9d=ecx*0xd597
   0x0000000000402697 <+119>:   mov    %r13,%r8     将 context 的值存给 r8
   0x000000000040269a <+122>:   shr    $0x20,%rsi      将 challenge 右移 32 位
   0x000000000040269e <+126>:   shr    $0x20,%r8       将 context 值右移 32 位
   0x00000000004026a2 <+130>:   xor    %r12d,%esi     esi^=challenge
   0x00000000004026a5 <+133>:   xor    %r13d,%r8d     r8d^=context
   0x00000000004026a8 <+136>:   shr    $0x10,%r13      将 context 值右移 16 位
   0x00000000004026ac <+140>:   mov    %esi,%edi       将 esi 的值存入 edi
   0x00000000004026ae <+142>:   xor    %r13d,%r9d     r9d^=r13d
   0x00000000004026b1 <+145>:   xor    %r8d,%edi     edi^=r8d
   0x00000000004026b4 <+148>:   call   0x402600 <secret_mix>      调用函数
Dump of assembler code for function secret_mix:
   0x0000000000402600 <+0>:     endbr64
   0x0000000000402604 <+4>:     mov    %edi,%eax     将 edi 存入 eax
   0x0000000000402606 <+6>:     rol    $1,%eax     eax 循环左移 1 位
   0x0000000000402608 <+8>:     add    $0xd597,%eax       eax+=0xd597
   0x000000000040260d <+13>:    xor    $0xd9ed,%eax     eax^=0xd9ed
   0x0000000000402612 <+18>:    ret
End of assembler dump.
   0x00000000004026b9 <+153>:   xor    %r9d,%eax      eax^=r9d
   0x00000000004026bc <+156>:   cmp    %eax,-0x28(%rbp)
   0x00000000004026bf <+159>:   jne    0x40264a <secret_phase+42>

   0x00000000004026c1 <+161>:   shr    $1,%esi     将 esi 的值右移 1 位
   0x00000000004026c3 <+163>:   lea    0xd9ed(%r8,%rcx,1),%edx     edx=0xd9ed+r8+rcx
   0x00000000004026cb <+171>:   add    %esi,%edx       edx+=esi
   0x00000000004026cd <+173>:   xor    %edx,%eax      eax^=edx
   0x00000000004026cf <+175>:   cmp    -0x24(%rbp),%eax     比较
   0x00000000004026d2 <+178>:   sete   %dl
   0x00000000004026d5 <+181>:   jmp    0x40264a <secret_phase+42>
```

推理过程

首先这个题目的关键是判断一下 r12 和 r13 哪个是 context，哪个是 challenge，我没找到好的方法，我写完代码两个都试了一遍，发现 r13 是 context，r12 是 challenge。接下来我们需要分析一下这个代码是干嘛的。从中我们发现，phase 主汇编中，出现了 cmp，jne 然后跳转爆炸。其实就是让我们算出来 3 个值，并和程序自己预留的答案进行比较，如果一样，就继续进行，不一样就爆炸了。所以整体反汇编的思路还是比较简单的。

我被卡了很长时间的地方是 secret_gate 部分对于数据格式的校验。首先第一个部分是对内容进行校验 call strlen ； sub $0x24, %rax ； rax = 长度 - 36 ； cmp $0x7f, %rax ; 这里表示说，rax 就是字符串的长度，长度要减去 36 并且小于 127，所以我们就知道，字符串长度杜比满足 36~127 之间，太长太短都会爆炸。

其次是对位置进行校验，我问了问豆包他跟我说这个东西叫魔数校验，主要是这两行汇编——cmpl $0x4e45504f, -0xb0(%rbp)；jne 0x40274b <失败分支> ，缓冲区的起始地址是 r12 里存的值，再往上找有一个 lea -0xd0(%rbq)，所以我们就知道缓冲区地址了，而在标黄这一行读的内存地址是-0xb0(%rbq)，所以相对偏移量是 32，也就意味着读取第 32 字节开始的连续 4 个字节，拿这四个字节和预期值进行比较，预期值就是标黄处的立即数。刚开始我没理解这个立即数，然后联想到前面一直在强调字符串长度，所以这里就想到了 ASCⅡ值，正好拆成四个字节，从小往大依次对应，就是大写的 OPEN。

接下来就是按照校验把那 3 个值包装成能过校验的形式就 ok 了。

使用公开列表以外的 seed 验证的记录

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

已完成关卡的运行结果截图

```
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```